50

QUESTIONS

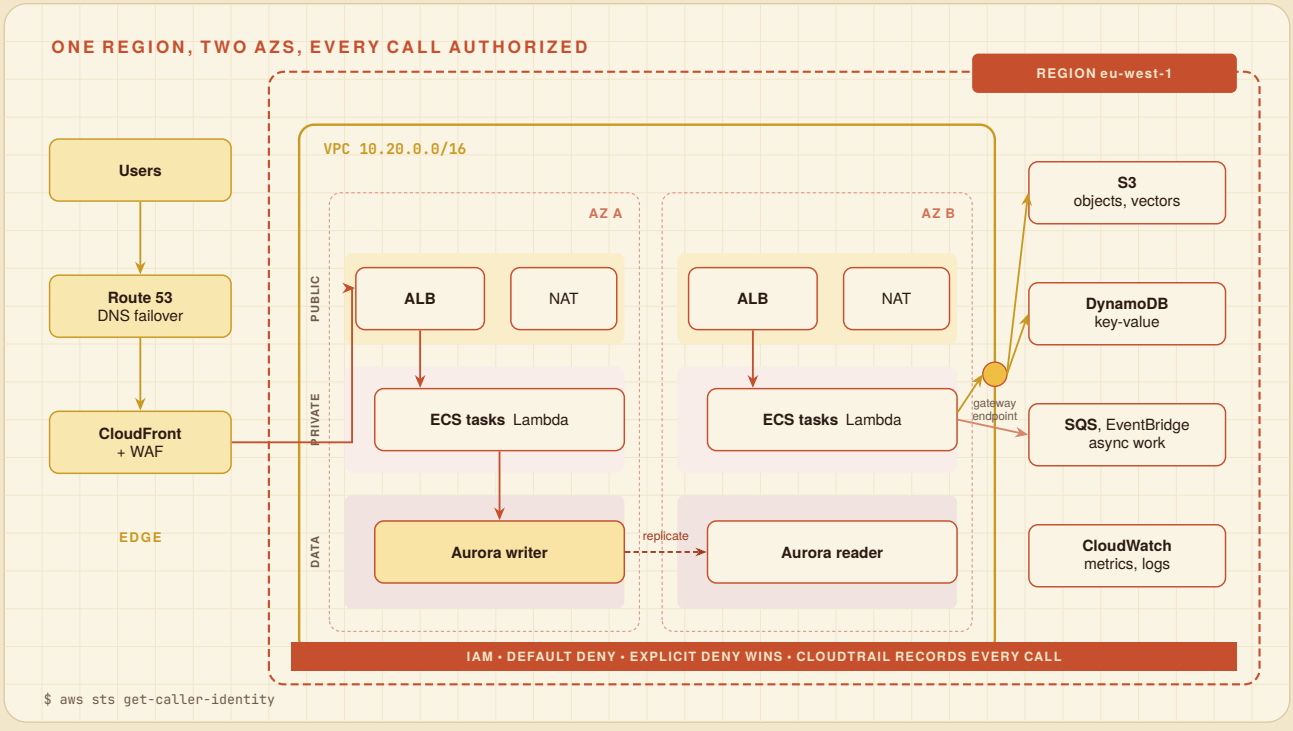
AWS

Interview Questions and Answers

Regions and AZs, IAM evaluation, compute and storage choices, VPC design, DynamoDB, Aurora, event-driven integration, security, deployment, observability, cost and production scenarios.

FUNDAMENTALS · COMPUTE & DATA · NETWORK & SECURITY · OPERATIONS · SCENARIOS

boto3 • Python



THE SIX PILLARS

every design answer is judged against them



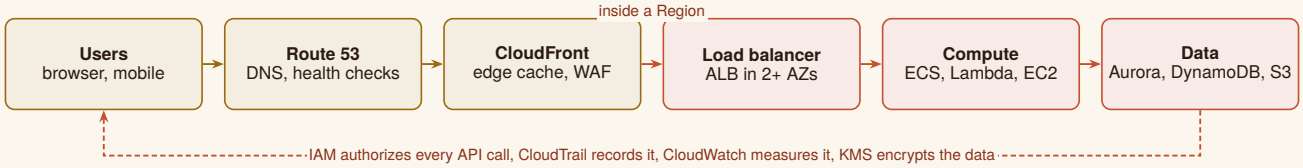
PART I

Introduction and Topic Overview

what, why, where it fits

Amazon Web Services (AWS) is a public cloud platform. It gives you compute, storage, databases, networking, security and AI services through an API. You pay for what you use. Each service is a managed building block. You connect the blocks to make a system. Every action is an API call. AWS Identity and Access Management (IAM) authorizes each call. AWS CloudTrail records it.

Why it matters. AWS has the largest share of the cloud market. Most backend, data and AI teams run some workloads on it. Interviewers do not test service names only. They test whether you can select the correct service, secure it, make it reliable and control its cost. This guide uses the service state of October 2026. It includes the releases from re:Invent 2025.



Where it fits. AWS is the run-time layer of many products. Your code runs on AWS compute. Your data stays in AWS storage. Your AI workloads call Amazon Bedrock or run on GPU instances. A good cloud engineer knows the limits of each service, the cost of each request path and the blast radius of each failure.

Use cases. Web APIs and SaaS backends; event-driven pipelines; data lakes and analytics; ML training and inference; generative AI with Bedrock; batch jobs; disaster recovery; hybrid networks to on-premises data centers.

Key concepts to know.

- **Global infrastructure**: Regions, AZs, edge locations.
- **IAM**: principals, policies, roles, evaluation logic.
- **Compute**: EC2, Lambda, ECS, EKS, Fargate.
- **Storage**: S3, EBS, EFS and their durability.
- **Databases**: RDS, Aurora, DynamoDB, ElastiCache.
- **Networking**: VPC, subnets, routes, endpoints.
- **Integration**: SQS, SNS, EventBridge, Step Functions.
- **Security**: KMS, Secrets Manager, GuardDuty.
- **Operations**: IaC, CI/CD, CloudWatch, X-Ray.
- **Cost**: pricing models, tags, data transfer.

II

PART II

Learning Roadmap

foundations to production

Level	What to learn	Why it matters	After this, you can build or explain
1. Foundations 3 to 4 weeks	Regions and AZs; the shared responsibility model; IAM users, roles and policies; MFA and the root user; the CLI and `aws configure sso`; EC2 instances, AMIs and security groups; S3 buckets, keys and storage classes; VPC basics; the Free Tier and AWS Budgets; the Well-Architected pillars.	Every AWS answer starts with "who can call which API, and in which Region." If you know IAM and the network basics, you can explain most access and connectivity problems.	A secure account with SSO and MFA; a static site on S3 and CloudFront; an EC2 web server in a public subnet; a budget alarm; an explanation of how IAM evaluates a request.
2. Practical engineering 6 to 8 weeks	Custom VPCs with public and private subnets; ALB and Auto Scaling groups; Lambda, API Gateway and DynamoDB; RDS and Aurora; SQS, SNS and EventBridge; Step Functions; boto3 with retries and paginators; CloudFormation or CDK; CloudWatch logs, metrics and alarms; Secrets Manager and KMS.	Real systems need private networks, managed data stores and asynchronous work. Interviewers check that you can wire services together with least privilege and that you can deploy them as code.	A serverless REST API with a DynamoDB table; a queue-based worker with a dead-letter queue; a three-tier app across two AZs; a full stack in CDK with a pipeline that deploys it.
3. Advanced and production ongoing	AWS Organizations, SCPs and RCPs; multi-account landing zones; Transit Gateway and PrivateLink; ECS and EKS at scale; DynamoDB single-table design; Aurora Global Database and DSQL; DR strategies; GuardDuty and Security Hub; cost allocation and Savings Plans; Bedrock, AgentCore and S3 Vectors.	Production problems are about scale, security incidents, outages and the monthly bill. These topics decide whether a platform stays safe, available and affordable as the company grows.	A multi-Region design with a known RTO and RPO; a diagnosis of an `AccessDenied` error in minutes; a 30% cost reduction with data; a RAG service on Bedrock with guardrails and evaluation.

KEY INSIGHT Learn IAM and VPC first. Most senior AWS questions reduce to two questions: which principal can call this API on this resource, and which network path does the request use? Then learn the cost of each path. Data transfer and NAT traffic often cost more than compute.

III

PART III

Technical Deep Dive

the subtopics that interviews probe

1. Global infrastructure

A Region is a separate geographic area. A Region has three or more Availability Zones (AZs). An AZ is one or more data centers with independent power and network. AZs in a Region connect with low-latency links. Edge locations serve CloudFront and Route 53. Most services are regional. IAM, Route 53 and CloudFront are global.

2. IAM evaluation logic

Step	Result
explicit `Deny` anywhere	request is denied
SCP or RCP does not allow	denied (org guardrail)
resource policy allows	allowed (same account)
identity policy allows	allowed, if the boundary and session policy also allow
nothing allows	implicit deny

3. The compute spectrum

Service	Use for	You manage
EC2	full control, GPUs	OS, patches, scaling
ECS + Fargate	containers, no servers	task definition
EKS	Kubernetes portability	cluster add-ons
Lambda	events, short tasks	code, memory
Lambda MI	steady Lambda load	instance choice

Lambda Managed Instances (re:Invent 2025) run Lambda functions on EC2 capacity in your account NEW.

4. Storage and databases

S3 stores objects with 11 nines of durability. EBS gives block volumes to one EC2 instance in one AZ. EFS gives a shared NFS file system across AZs. RDS and Aurora run relational engines. DynamoDB is a key-value store with single-digit millisecond latency at any scale. Aurora DSQL is a serverless, distributed SQL database with active-active multi-Region writes.

5. Networking

A VPC is your private network in one Region. A subnet is in one AZ. A route table sends traffic to an internet gateway, a NAT gateway, a VPC endpoint or a transit gateway. Security groups filter traffic at the network interface. VPC endpoints keep traffic to AWS services off the internet.

6. Event-driven integration

SQS buffers work for one consumer group. SNS pushes one message to many subscribers. EventBridge routes events by content. Kinesis keeps ordered records for replay. Step Functions runs workflows with state and retries.

7. Reliability by design

Put each tier in two or more AZs. Make compute stateless. Keep state in managed stores. Use queues to absorb spikes. Set timeouts, retries with jitter and circuit breakers. Test failure with AWS Fault Injection Service. Define the RTO and RPO before you select a DR strategy.

8. Security and audit

CloudTrail logs API calls. AWS Config records resource changes and checks rules. GuardDuty finds threats in logs and runtime activity. Security Hub collects findings. IAM Access Analyzer finds external access and unused permissions. KMS controls keys for encryption at rest.

9. The cost model

You pay for compute time, storage, requests and data transfer. Data into AWS is free. Data out to the internet, across AZs and through NAT gateways costs money. Savings Plans and Reserved Instances reduce the cost of steady load. Spot reduces the cost of interruptible work.

10. Recent releases

2025: SQS messages up to 1 MiB; Aurora DSQL GA; DynamoDB global tables with strong consistency; Bedrock AgentCore GA; Python 3.14 on Lambda. re:Invent 2025: Lambda durable functions; Lambda Managed Instances; S3 Vectors GA; 50 TB S3 objects; Database Savings Plans NEW.

IV QUESTIONS 1 TO 10 Fundamentals

infrastructure, IAM, core service choices

Q01 What are Regions, Availability Zones and edge locations? BASIC

A Region is a geographic area with its own isolated set of services. An AZ is a group of data centers in a Region with independent power, cooling and network. Edge locations are points of presence in many cities. CloudFront and Route 53 use them to serve users with low latency. **EXAMPLE** Run the app in `eu-west-1` across three AZs. Serve static files from CloudFront edge locations.

> **INSIGHT** Select a Region by four criteria: data residency law, latency to users, service availability and price.

Q02 What is the shared responsibility model? BASIC

AWS secures the cloud: data centers, hardware, the network and the hypervisor. You secure what you put in the cloud: your data, IAM, network rules, the guest OS and encryption settings. The split moves with the service. On EC2, you patch the OS. On Lambda and DynamoDB, AWS patches the platform. **EXAMPLE** An open S3 bucket is a customer failure. A failed disk in S3 is an AWS problem.

> **INSIGHT** Managed services move more responsibility to AWS. This is often the best security control.

Q03 How does IAM decide if a request is allowed? BASIC

By default, IAM denies every request. IAM then checks all policies that apply. An explicit `Deny` in any policy wins. Organization SCPs and RCPs set the maximum permissions. Then an identity-based or resource-based policy must allow the action. Permissions boundaries and session policies can only reduce access. **EXAMPLE** An admin role cannot delete a bucket if an SCP denies `s3:DeleteBucket`.

> **INSIGHT** Say "default deny, explicit deny wins, then an allow is necessary." Interviewers listen for this order.

Q04 Why do you use IAM roles and not access keys? BASIC

An IAM user has long-term credentials. Keys can leak in code, logs or Git. A role gives temporary credentials from AWS STS. The credentials expire automatically. EC2, Lambda and ECS get role credentials through their metadata service. People sign in through IAM Identity Center. **EXAMPLE** Attach a role to a Lambda function. boto3 finds its credentials. You do not write keys in code.

> **INSIGHT** Use IMDSv2 on EC2. It requires a session token and blocks most SSRF attacks on credentials.

Q05 How do identity-based and resource-based policies differ? BASIC

You attach an identity-based policy to a user, group or role. It says what this principal can do. You attach a resource-based policy to a resource, for example an S3 bucket or an SQS queue. It says which principals can use the resource. For cross-account access, both sides must allow the request, or the caller assumes a role in the other account with `sts.assume_role()`.

> **INSIGHT** For a third-party role, require an `ExternalId`. It prevents the confused deputy problem.

Q06 How do you set up and secure AWS accounts? BASIC

Use AWS Organizations with one account for each workload and environment. Accounts are the strongest isolation boundary. Use AWS Control Tower for a landing zone. Lock the root user: add MFA, remove access keys and do not use it for daily work. Use SCPs to block risky actions, for example disabling CloudTrail or using unapproved Regions. **EXAMPLE** Separate accounts: `prod`, `staging`, `dev`, `security`, `log-archive`.

INSIGHT Send CloudTrail logs to a separate log-archive account. An attacker in prod cannot delete them.

Q07 What is an ARN, and why are tags important? BASIC

An Amazon Resource Name identifies one resource: `arn:partition:service:region:account:resource`. Policies use ARNs to scope access. Tags are key-value pairs on resources. Use them for cost allocation, automation and attribute-based access control (ABAC). **EXAMPLE** ABAC: allow `ec2:StopInstances` only if `aws:ResourceTag/team` equals `aws:PrincipalTag/team`.

INSIGHT Enforce required tags with tag policies and SCPs. Untagged resources make cost reports useless.

Q08 How do you select EC2, Lambda, ECS or EKS? BASIC

Use Lambda for event-driven tasks that finish in 15 minutes or less. Use ECS on Fargate for containers without server management. Use EKS when you need Kubernetes APIs or portability. Use EC2 when you need full control, special hardware or steady high load at the lowest unit cost. **EXAMPLE** An image-resize trigger on S3: Lambda. A long-running API in containers: ECS on Fargate.

INSIGHT Start with the most managed option that meets the requirements. Move down only for a measured reason.

Q09 Compare S3, EBS, EFS and instance store. BASIC

S3 is object storage over HTTPS. It is regional and very durable. EBS is a network block volume for one instance in one AZ. Use snapshots for backup. EFS is a shared file system for many instances across AZs. Instance store is a local disk. You lose its data when the instance stops. **EXAMPLE** Database data files: EBS `gp3` or `io2`. Shared uploads: EFS. Data lake: S3.

INSIGHT `gp3` lets you set IOPS and throughput independently of size. It is usually cheaper than `gp2`.

Q10 What are the EC2 pricing models? BASIC

On-Demand: pay by the second, no commitment. Savings Plans: commit to a spend per hour for one or three years, up to 72% off. Reserved Instances: a similar discount for a specified instance family. Spot: unused capacity, up to 90% off, but AWS can stop it with a two-minute notice. **EXAMPLE** Steady baseline on a Savings Plan, peaks on On-Demand, batch jobs on Spot.

INSIGHT Graviton (Arm) instances often give the best price performance. Test your code on Arm first.

V QUESTIONS 11 TO 20
Compute, Storage and Data

scaling, S3, DynamoDB, relational, caching

Q11 How does an EC2 Auto Scaling group work? MID

An Auto Scaling group (ASG) uses a launch template to start instances in many AZs. It keeps the count between a minimum and a maximum. A target tracking policy adds or removes instances to keep a metric near a target. The ASG replaces unhealthy instances. Use ELB health checks, not only EC2 status checks. **EXAMPLE** Target tracking: keep average CPU at 50%. Add a warm pool to reduce start time.

INSIGHT Use instance refresh to roll out a new AMI. It replaces instances in batches and checks health.

Q12 How does Lambda scale, and what is a cold start? MID

Each concurrent request needs one execution environment. A new environment downloads code, starts the runtime and runs your init code. This is a cold start. Each function can add 1,000 concurrent environments every 10 seconds. The default account limit is 1,000 concurrent executions in each Region. **EXAMPLE** Make SDK clients outside the handler. The init phase runs one time for each environment.

INSIGHT To reduce cold starts, use SnapStart (Python 3.12+) or provisioned concurrency. Since August 2025, AWS bills the init phase.

Q13 Explain S3 storage classes and lifecycle rules. MID

S3 Standard is for frequent access. Standard-IA and One Zone-IA cost less to store but add a retrieval fee. Glacier Instant, Flexible and Deep Archive are for archives. Intelligent-Tiering moves objects between tiers by access pattern. Lifecycle rules move or delete objects by age. **EXAMPLE** Logs: Standard for 30 days, then Glacier Flexible Retrieval, then delete after 400 days.

INSIGHT Small objects have minimum size and duration charges in IA tiers. Check object size before you move them.

Q14 What are the consistency and performance rules of S3? MID

S3 gives strong read-after-write consistency for all PUT, DELETE and LIST operations. Each partitioned prefix supports 3,500 write and 5,500 read requests per second. Use many prefixes to scale. Use multipart upload for objects larger than 100 MB. The maximum object size is 50 TB since December 2025 NEW. **EXAMPLE** Write keys as `tenant=42/date=2026-10-13/part-0001.parquet`. Readers can then select by prefix.

INSIGHT S3 has no rename. A "rename" is a copy and a delete. Design key names before you write data.

Q15 How do you secure an S3 bucket? MID

Keep Block Public Access on. It is on by default for new buckets. Keep ACLs disabled with "bucket owner enforced." S3 encrypts new objects with SSE-S3 by default. Use SSE-KMS when you need key control and audit. Deny requests without TLS with `aws:SecureTransport`. Give temporary access with presigned URLs.

```python
url = boto3.client("s3").generate_presigned_url(
    "get_object",
    Params={"Bucket": "reports", "Key": "q3.pdf"},
    ExpiresIn=900)  # 15 minutes
```

INSIGHT Serve public content through CloudFront with Origin Access Control. Keep the bucket private.

Q16 How do you design a DynamoDB table?

MID

Start with the access patterns, not with entities. The partition key spreads data across partitions. The sort key orders items in a partition and supports range queries. Put related items in one partition to read them with one `Query`. Add a global secondary index (GSI) for each extra access pattern.

```python
from boto3.dynamodb.conditions import Key
table = boto3.resource("dynamodb").Table("app")
resp = table.query(
    KeyConditionExpression=Key("PK").eq("CUST#42")
    & Key("SK").begins_with("ORDER#2026-"))
```

INSIGHT Do not use `Scan` in a request path. It reads the full table and costs capacity for each item.

Q17 Explain DynamoDB capacity modes and consistency.

MID

On-demand mode bills each request and scales automatically. Provisioned mode sets read and write capacity, with auto scaling. Reads are eventually consistent by default. A strongly consistent read costs twice as much. Transactions write up to 100 items atomically. Global tables replicate across Regions. Since 2025, they also support multi-Region strong consistency. **EXAMPLE** Spiky new workload: on-demand. Steady, predictable traffic: provisioned with auto scaling.

INSIGHT A hot partition key throttles the table even with spare capacity. Add a suffix to spread the writes.

Q18 When do you use RDS, Aurora, DynamoDB or Aurora DSQL?

MID

Use RDS for a standard managed engine: PostgreSQL, MySQL, SQL Server or Oracle. Use Aurora for higher throughput, fast failover and up to 15 read replicas on shared storage. Use DynamoDB for known key-based access at any scale. Use Aurora DSQL for serverless PostgreSQL-compatible SQL with active-active writes in many Regions. **EXAMPLE** Order system with joins and reports: Aurora PostgreSQL. Session store: DynamoDB with TTL.

INSIGHT DSQL uses optimistic concurrency. Your code must retry transactions that fail with conflicts.

Q19 What is the difference between Multi-AZ and read replicas?

MID

A Multi-AZ deployment keeps a synchronous standby in another AZ. It is for availability. If the primary fails, RDS moves the DNS name to the standby. A read replica uses asynchronous replication. It is for read scale. It can lag. You can promote it manually, for example for cross-Region DR. **EXAMPLE** Multi-AZ DB cluster: two readable standbys and failover in about 35 seconds.

INSIGHT Your app must reconnect after failover. Use RDS Proxy to keep connections and to shorten failover.

Q20 How do you add caching on AWS?

MID

Use CloudFront to cache content at the edge. Use ElastiCache (Valkey, Redis OSS or Memcached) for an in-memory cache in front of a database. Use DAX for microsecond reads from DynamoDB. Select a pattern: cache-aside for reads, write-through for data that you read soon after you write it. Set a TTL on every key. **EXAMPLE** Cache a product page in ElastiCache for 60 seconds. Invalidate the key when the price changes.

INSIGHT ElastiCache for Valkey costs less than Redis OSS. It uses the same commands.

VI QUESTIONS 21 TO 30 Networking and Security

VPC, load balancing, DNS, keys, secrets, detection

Q21 How do you design a VPC?

MID

Select a CIDR block that does not overlap with other networks, for example `10.20.0.0/16`. Make public and private subnets in two or three AZs. Public subnets route to an internet gateway. Private subnets route to a NAT gateway for outbound access. Put load balancers in public subnets. Put compute and databases in private subnets. **EXAMPLE** Three tiers in three AZs: nine subnets. AWS reserves five IP addresses in each subnet.

INSIGHT Make one NAT gateway in each AZ. One shared NAT gateway is a single point of failure.

Q22 What is the difference between security groups and NACLs?

MID

A security group is attached to a network interface. It is stateful: return traffic is allowed automatically. It has allow rules only. A network ACL is attached to a subnet. It is stateless: you must allow return traffic on ephemeral ports. It has numbered allow and deny rules. **EXAMPLE** Database SG: allow port 5432 only from the app security group ID, not from a CIDR.

INSIGHT Use security groups for most control. Use NACLs only for broad subnet-level deny rules.

Q23 What are VPC endpoints, and why do you use them?

MID

A VPC endpoint connects your VPC to an AWS service without the internet. A gateway endpoint (S3 and DynamoDB) is a route table entry. It is free. An interface endpoint (PrivateLink) is a network interface with a private IP. It has an hourly and a per-GB price. Endpoint policies can limit access to specified buckets. **EXAMPLE** Private Lambda functions read S3 through a gateway endpoint, not through the NAT gateway.

INSIGHT Gateway endpoints for S3 often remove the largest part of a NAT gateway bill.

Q24 How do you connect many VPCs and on-premises networks?

SENIOR

VPC peering connects two VPCs. It is not transitive. Transit Gateway is a regional hub for many VPCs and VPNs, with route tables for segmentation. PrivateLink exposes one service, not a full network. AWS Cloud WAN manages a global network with policies. Site-to-Site VPN and Direct Connect connect data centers. **EXAMPLE** 50 VPCs and two data centers: Transit Gateway with Direct Connect and a VPN backup.

> **INSIGHT** Plan non-overlapping CIDRs early. Overlap blocks peering and Transit Gateway routing.

Q25 Compare ALB, NLB and Gateway Load Balancer.

MID

An Application Load Balancer works at layer 7. It routes by host, path and header, and it supports HTTP/2, gRPC and WebSockets. A Network Load Balancer works at layer 4. It handles TCP, UDP and TLS with very low latency and static IPs. A Gateway Load Balancer sends traffic through firewall appliances. **EXAMPLE** REST API with path routing: ALB. Game server on UDP: NLB.

> **INSIGHT** Attach AWS WAF to an ALB, API Gateway or CloudFront to block common web attacks.

Q26 What are the Route 53 routing policies?

MID

Simple returns one record. Weighted splits traffic by percentage. Latency sends users to the Region with the lowest latency. Failover sends traffic to a secondary when a health check fails. Geolocation and geoproximity route by user location. Multivalue returns up to eight healthy records. **EXAMPLE** Canary for a new Region: weighted 95/5, then increase the weight when the metrics are good.

> **INSIGHT** DNS failover depends on TTL. Set a short TTL, for example 60 seconds, on failover records.

Q27 How does CloudFront work with a private origin?

MID

CloudFront caches content at edge locations. A cache miss goes to the origin: S3, an ALB or a custom HTTP server. Origin Access Control (OAC) lets CloudFront sign requests to S3. The bucket policy allows only that distribution. Use signed URLs or signed cookies for private content. **EXAMPLE** Static site: S3 with OAC, a cache policy with a long TTL and versioned file names.

> **INSIGHT** Use versioned file names, not invalidations, to release new assets. Invalidations are slow and can cost money.

Q28 How does KMS envelope encryption work?

SENIOR

KMS keeps the key material in HSMs. The key never leaves KMS unencrypted. The KMS `Encrypt` API accepts only 4 KB. For larger data, KMS makes a data key. You encrypt the data locally with the plaintext data key. Then you store the encrypted data key with the data and delete the plaintext key.

PYTHON

```python
dk = boto3.client("kms").generate_data_key(
    KeyId="alias/app", KeySpec="AES_256")
blob = aes_gcm_encrypt(dk["Plaintext"], data)
store(blob, dk["CiphertextBlob"])
del dk  # do not keep the plaintext key
```

> **INSIGHT** The key policy is the main control. If the key policy does not allow a principal, IAM alone cannot.

Q29 Secrets Manager or Parameter Store?

MID

Both store configuration securely. Secrets Manager adds automatic rotation, cross-Region replication and native integration with RDS. It costs a fee for each secret. Parameter Store is free for standard parameters. It supports `SecureString` with KMS. Use it for configuration and secrets that do not rotate. **EXAMPLE** `get_secret_value(SecretId="prod/db")` returns the secret as a JSON string.

> **INSIGHT** Cache secrets in memory with a short TTL. One API call for each request adds latency and cost.

Q30 How do you detect and investigate threats?

SENIOR

Enable CloudTrail in all Regions with an organization trail. Enable GuardDuty in all accounts. It finds threats from CloudTrail, VPC Flow Logs, DNS logs and runtime agents. Security Hub collects findings and checks controls. AWS Config records resource changes. Amazon Detective helps you investigate a finding. **EXAMPLE** GuardDuty finding: EC2 role credentials used from an external IP. Revoke sessions and rotate.

> **INSIGHT** Automate the first response with EventBridge rules: isolate the instance, then alert a human.

QUESTIONS 31 TO 40

VII Serverless, Containers and Operations

messaging, IaC, deploys, cost

Q31 When do you use SQS, SNS, EventBridge or Kinesis?

MID

Use SQS to buffer work for one consumer group. Use SNS to push one message to many subscribers. Use EventBridge to route events by content, from AWS services, SaaS partners or your apps. Use Kinesis Data Streams for ordered records that many consumers read and replay. **EXAMPLE** Order placed: EventBridge routes it to billing, email and analytics. Each target has its own SQS queue.

INSIGHT SNS plus SQS (fan-out) gives each consumer its own retries and its own dead-letter queue.

Q32 How do you process SQS messages safely with Lambda?

MID

Standard queues deliver at least once and do not keep strict order. FIFO queues keep order in each message group and remove duplicates. Set the visibility timeout to six times the function timeout. Send failed messages to a dead-letter queue. Return only the failed items, so the batch does not run again.

PYTHON

```python
def handler(event, context):
    failed = []
    for rec in event["Records"]:
        try:
            process(json.loads(rec["body"]))
        except Exception:
            failed.append(
                {"itemIdentifier": rec["messageId"]})
    return {"batchItemFailures": failed}
```

INSIGHT Enable `ReportBatchItemFailures` on the event source mapping. Without it, Lambda ignores this response.

Q33 How do you make a Lambda function idempotent?

SENIOR

Events can arrive more than one time. Make a stable idempotency key from the request, for example an order ID. Before the work, write the key with a conditional put. If the key exists, return the saved result. Add a TTL to delete old keys. Powertools for AWS Lambda has a ready idempotency utility.

PYTHON

```python
try:
    table.put_item(
        Item={"PK": f"IDEMP#{key}", "ttl": exp},
        ConditionExpression="attribute_not_exists(PK)
    ")
except ddb.exceptions.ConditionalCheckFailedException:

    return saved_result(key)  # duplicate request
```

INSIGHT Idempotency protects you from retries in SQS, EventBridge, API clients and your own code.

Q34 Step Functions or Lambda durable functions?

SENIOR

Step Functions defines a workflow as a state machine with retries, branches and service integrations. Standard workflows run up to one year. Express workflows run up to five minutes at high volume. Lambda durable functions (re:Invent 2025) let you write the workflow as code in Python or TypeScript NEW. The function saves checkpoints and can wait up to one year without compute cost. **EXAMPLE** Visual approval flow across many services: Step Functions. Multi-step agent logic in code: durable functions.

INSIGHT Both replay from saved state. Keep steps deterministic and side effects idempotent.

Q35 ECS or EKS, and Fargate or EC2?

MID

ECS is the AWS container orchestrator. It is simple and has deep AWS integration. EKS runs Kubernetes. Use it for Kubernetes tools, Helm charts or multi-cloud portability. Fargate runs tasks without servers. The EC2 launch type gives GPUs, daemon sets and lower cost at high utilization. EKS Auto Mode manages nodes for you. **EXAMPLE** Small team with ten services: ECS on Fargate. Platform team with Kubernetes skills: EKS.

INSIGHT Use IAM roles for tasks (ECS) or EKS Pod Identity. Do not share one node role for all pods.

Q36 How do you manage infrastructure as code?

MID

Use CloudFormation, the AWS CDK or Terraform. Keep templates in Git. Review a change set or a plan before you deploy. Detect drift on a schedule. With the CDK, you write Python and it makes CloudFormation templates. Grant methods make least-privilege policies.

PYTHON

```python
class Api(Stack):
    def __init__(self, scope, id, **kw):
        super().__init__(scope, id, **kw)
        t = ddb.TableV2(self, "T", partition_key=ddb.
    Attribute(
            name="PK", type=ddb.AttributeType.STRING)
    )
        fn = lambda_.Function(self, "Fn", handler="
    app.handler",
            runtime=lambda_.Runtime.PYTHON_3_13,
            code=lambda_.Code.from_asset("src"))
        t.grant_read_write_data(fn)
```

INSIGHT Set a deletion policy of `RETAIN` on databases and buckets. A stack delete must not delete data.

Q37 How do you deploy safely?

MID

Use a pipeline: build, test, deploy to staging, then deploy to production. Use blue/green or canary releases. For Lambda, shift traffic between alias versions with CodeDeploy. For ECS, use the native blue/green deployment. Roll back automatically when a CloudWatch alarm goes into the `ALARM` state. **EXAMPLE** `Canary10Percent5Minutes`: 10% of traffic for five minutes, then 100% if alarms stay OK.

INSIGHT Deploy one Region and one AZ at a time. This limits the blast radius of a bad change.

Q38 How do you observe an AWS workload?

MID

Send structured JSON logs to CloudWatch Logs. Query them with Logs Insights. Publish metrics with the Embedded Metric Format.

Make alarms on symptoms: error rate, latency p99 and queue age. Trace requests with X-Ray or the AWS Distro for OpenTelemetry. CloudWatch Application Signals shows service-level objectives. **EXAMPLE** Alarm on `ApproximateAgeOfOldestMessage` for SQS. It shows that consumers fall behind.

INSIGHT Set a log retention period on every log group. The default is "never expire," and storage grows.

Q39 What are the boto3 best practices? MID

Make clients one time and reuse them. Clients are thread-safe. Resources and sessions are not. Configure retries and timeouts explicitly. Use paginators for list APIs and waiters for long operations. Catch `ClientError` and read the error code.

```python
cfg = Config(retries={"mode": "adaptive", "
     max_attempts": 8},
            connect_timeout=3, read_timeout=10)
s3 = boto3.client("s3", config=cfg)
pages = s3.get_paginator("list_objects_v2").paginate(
    Bucket="logs", Prefix="2026/10/")
for obj in (o for p in pages for o in p.get("Contents
     ", [])):
    handle(obj["Key"])
```

INSIGHT A list call returns 1,000 keys at most. Without a paginator, your code silently misses data.

Q40 How do you reduce the AWS bill? MID

First, make costs visible: activate cost allocation tags and use Cost Explorer and AWS Budgets. Then remove waste: idle resources, unattached EBS volumes and old snapshots. Rightsize with Compute Optimizer. Move to Graviton. Buy Savings Plans for the steady baseline. Use Spot for batch. Reduce NAT and cross-AZ data transfer. **EXAMPLE** Database Savings Plans (December 2025) give up to 35% off database usage NEW.

INSIGHT Give each team a monthly cost report by tag. Teams reduce what they can see.

VII QUESTIONS 41 TO 50
Advanced and Scenario-Based

design, diagnosis, failure analysis

Q41 Design a highly available web application. SENIOR

Use Route 53 and CloudFront in front. Put an ALB across three AZs. Run stateless containers on ECS in an Auto Scaling service. Keep sessions in ElastiCache or DynamoDB. Use Aurora with a writer and a reader in different AZs. Put files in S3. Send slow work to SQS and workers. **EXAMPLE** One AZ fails: the ALB stops sending traffic to it, ECS starts tasks in the other AZs, Aurora fails over.

INSIGHT Size each AZ to carry the full load when one AZ fails. With three AZs, run at 150% of the need.

Q42 Explain the disaster recovery strategies. SENIOR

Backup and restore: lowest cost, RTO in hours. Pilot light: data replicates and the core stays ready, RTO in tens of minutes. Warm standby: a smaller full copy runs, RTO in minutes. Multi-site active-active: full capacity in two Regions, RTO near zero. RPO depends on the replication type. **EXAMPLE** Aurora Global Database: replication lag usually under one second and cross-Region failover in minutes.

INSIGHT Test the failover. A DR plan that you did not test is only a document.

Q43 A traffic spike causes Lambda throttles. What do you do? SENIOR

Read the `Throttles` and `ConcurrentExecutions` metrics. Check the account concurrency limit and any reserved concurrency on the function. Put SQS in front of the function to absorb the spike. Set the maximum concurrency on the SQS event source to protect the database. Request a quota increase if the load is real. **EXAMPLE** 1,000 functions share one account limit. One busy function can throttle all the others.

INSIGHT Reserved concurrency protects critical functions and also limits its noisy ones.

Q44 An instance in a private subnet cannot reach S3. How do you find the cause? SENIOR

Check the layers in order. Route table: is there a route to a NAT gateway or an S3 gateway endpoint? Security group: is outbound HTTPS allowed? NACL: are the return ports allowed? Endpoint policy and bucket policy: do they allow the access? IAM: does the role allow the action? DNS: is DNS resolution on in the VPC? **EXAMPLE** Reachability Analyzer tests the network path. CloudTrail shows if the request arrived at S3.

INSIGHT A timeout points to the network. An `AccessDenied` error points to a policy.

Q45 How do you debug an AccessDenied error? SENIOR

Read the full error. It often names the policy type that denied the request. Find the call in CloudTrail and check the principal, action, resource and Region. Check SCPs, permissions boundaries, resource policies, KMS key policies and VPC endpoint policies. Use the IAM policy simulator. Decode encoded messages with STS. **EXAMPLE** `aws sts decode-authorization-message --encoded-message <msg>` shows the failed context.

INSIGHT A missing `kms:Decrypt` permission often causes an S3 `AccessDenied` on SSE-KMS objects.

Q46 The NAT gateway cost increases fast. How do you fix it?

SENIOR

Find the traffic. Enable VPC Flow Logs and group the bytes by source and destination. Cost Explorer shows the `NatGateway-Bytes` usage type. Common causes are S3 or DynamoDB traffic without gateway endpoints, image pulls from public registries and cross-AZ routing to a NAT gateway. **EXAMPLE** Add S3 and DynamoDB gateway endpoints. Use ECR pull-through cache. Use one NAT gateway in each AZ.

INSIGHT NAT gateways charge for each GB processed. A gateway endpoint for S3 is free.

Q47 Process 100 million objects in S3.

SENIOR

Do not list the bucket in a loop. Use S3 Inventory to get a manifest of all keys. Then use S3 Batch Operations to run a Lambda function or a copy for each object. For complex logic, use the Step Functions Distributed Map. It reads the manifest and runs up to 10,000 parallel child workflows. **EXAMPLE** Re-encrypt all objects with a new KMS key: S3 Batch Operations with a copy job and a report.

INSIGHT Set limits on concurrency. 10,000 parallel workers can overload a downstream database.

Q48 How do you isolate tenants in a SaaS on AWS?

SENIOR

Select a model. Silo: one account or stack for each tenant, strong isolation, high cost. Pool: shared resources with a tenant ID on each item and IAM conditions. Bridge: mix both by tier. Use ABAC with tenant tags and DynamoDB `LeadingKeys` conditions. Lambda tenant isolation mode (November 2025) gives each tenant separate execution environments NEW. **EXAMPLE** Premium tenants in silo accounts; free tenants in a pooled stack.

INSIGHT Put the tenant ID in the token and in every log line. You need it for cost and for incidents.

Q49 Build a RAG application on AWS.

SENIOR

Store documents in S3. Use Bedrock Knowledge Bases to chunk, embed and index them. Store vectors in S3 Vectors (GA December 2025) for low cost, or in OpenSearch for low latency. Call models with the Bedrock Converse API. Add Bedrock Guardrails. Use Bedrock AgentCore to run agents with memory, tools and identity.

PYTHON

```
br = boto3.client("bedrock-runtime")
r = br.converse(modelId=MODEL_ID,
    system=[{"text": "Use only the context."}],
    messages=[{"role": "user", "content": [
        {"text": f"{ctx}\n\nQuestion: {q}"}]}],
    inferenceConfig={"maxTokens": 512,
                     "temperature": 0})
answer = r["output"]["message"]["content"][0]["text"]
```

INSIGHT Measure retrieval quality and answer faithfulness before launch. Bedrock has built-in evaluation jobs.

Q50 How do you migrate a monolith to AWS?

SENIOR

Assess each application with the 7 Rs: retire, retain, rehost, relocate, repurchase, replatform and refactor. Move the monolith first with low change, for example rehost or replatform. Use AWS DMS to move databases with ongoing replication. Then extract services one at a time with the strangler fig pattern. **EXAMPLE** Week 1: rehost on EC2. Month 3: database on Aurora. Month 6: extract the billing service to ECS.

INSIGHT Measure before and after: latency, error rate and cost. A migration without numbers cannot prove value.

IX PART IX Cheat Sheet

the page to revisit

CLI commands

Command	Purpose
`aws sso login`	sign in with Identity Center
`aws sts get-caller-identity`	which principal am I?
`aws s3 sync ./site s3://b`	upload changed files
`aws logs tail <group> --follow`	live logs
`aws sts decode-authorization-message`	denied context
`aws iam simulate-principal-policy`	test policy
`aws cloudformation deploy`	deploy a template
`cdk diff` / `cdk deploy`	review, then deploy CDK
`aws ce get-cost-and-usage`	cost by service or tag

Select the compute

Need	Use
event, under 15 min	Lambda
long workflow in code	Lambda durable functions
containers, no servers	ECS on Fargate
Kubernetes APIs	EKS (Auto Mode)
GPU, full control	EC2
interruptible batch	EC2 Spot, AWS Batch

Select the data store

Need	Use
key-value at any scale	DynamoDB
relational, joins	Aurora, RDS
multi-Region SQL writes	Aurora DSQL
cache, sessions	ElastiCache (Valkey)
objects, data lake	S3
vectors at low cost	S3 Vectors
search, low-latency vectors	OpenSearch

Numbers to remember

Item	Value
Lambda timeout, memory	15 min, 128 MB to 10 GB
Lambda payload	6 MB sync, 1 MB async
Lambda concurrency (default)	1,000 per Region
SQS message, retention	1 MiB, up to 14 days
SQS visibility timeout	30 s default, 12 h max
S3 object, single PUT	50 TB, 5 GB
S3 per prefix	3,500 writes, 5,500 reads/s
DynamoDB item	400 KB
DynamoDB transaction	100 items
Step Functions Standard	up to 1 year
Step Functions Express	up to 5 min
KMS `Encrypt` plaintext	4 KB
Subnet reserved IPs	5
VPC CIDR size	/16 to /28
Spot interruption notice	2 min

Pitfalls

- Long-term access keys in code or Git.
- Root user without MFA or with access keys.
- One NAT gateway for all AZs.
- S3 traffic through NAT, not an endpoint.
- `Scan` in a DynamoDB request path.
- List calls without paginators.
- Visibility timeout shorter than the work.
- Log groups with no retention period.
- No deletion policy on stateful resources.
- A DR plan that nobody tested.

PART X

Implementation Checklist and Key Takeaways

apply it, then go deeper

Practical checklist

- ☐ Use Organizations with separate accounts for each environment.
- ☐ Lock the root user with MFA and no access keys.
- ☐ Use roles and Identity Center. Use no long-term keys.
- ☐ Enable CloudTrail, GuardDuty and Config in all Regions.
- ☐ Keep S3 Block Public Access on.
- ☐ Run each tier in two or more AZs.
- ☐ Add S3 and DynamoDB gateway endpoints.
- ☐ Put a dead-letter queue on every queue and async target.
- ☐ Make consumers idempotent.
- ☐ Deploy all resources as code, with review.
- ☐ Make alarms on error rate, p99 latency and queue age.
- ☐ Tag every resource with owner, team and environment.

Build milestones

1. **Static site**: S3, CloudFront with OAC, Route 53.
2. **Serverless API**: API Gateway, Lambda, DynamoDB, CDK.
3. **Async worker**: SQS, DLQ, idempotent Lambda.
4. **Three-tier app**: VPC, ALB, ECS, Aurora, three AZs.
5. **Observability**: logs, metrics, traces, alarms.
6. **GenAI**: Bedrock, Knowledge Bases, S3 Vectors.

Principles to remember

- Every action is an API call. IAM decides each one.
- Default deny. An explicit deny always wins.
- Design for AZ failure from the first day.
- Use the most managed service that meets the need.
- Know the cost of each request path, including data transfer.

Explore next

Well-Architected lenses; the Amazon Builders' Library; DynamoDB single-table design; Transit Gateway and Cloud WAN; EKS at scale; Bedrock AgentCore; AWS Fault Injection Service.

What you can now do

Explain Regions, AZs and the shared responsibility model; reason about IAM evaluation; select compute, storage and messaging for each workload; design a secure VPC; build serverless and container systems as code; and diagnose access, network, scaling and cost problems.

References

AWS documentation (docs.aws.amazon.com) for IAM, VPC, S3, Lambda, DynamoDB and SQS. AWS Well-Architected Framework. AWS What's New, 2025 and 2026. Boto3 and AWS CDK documentation.

Keep going with AI Engineering Insider

Practitioner-grade guides on LLMs, RAG, inference and agents. aiengineeringinsider.substack.com/subscribe • beacons.ai/aiengineeringinsider